PROYECTOS EN TECNOLOGÍA

Organización GitHub y GitHub Projects

Geoffrey Hecht

GITHUB ORGANIZATIONS

- En este curso trabajaremos en una **organización de GitHub**, `TECH-MACI`, que agrupa los repositorios, los equipos y los proyectos del curso
- Cada grupo `2026-GXX` tiene:
  - un **repositorio** privado, creado desde la **plantilla de repositorio** (*template*) `PROJECT-BASE`: datos del proyecto y plantillas de issues para historias de usuario y tareas
  - un **equipo** (*team*), que da acceso al repositorio
  - un **proyecto** `2026-GXX-project`, creado desde la plantilla `[TEMPLATE] Scrum project`
- Proyectos: github.com/orgs/TECH-MACI/projects

ISSUES

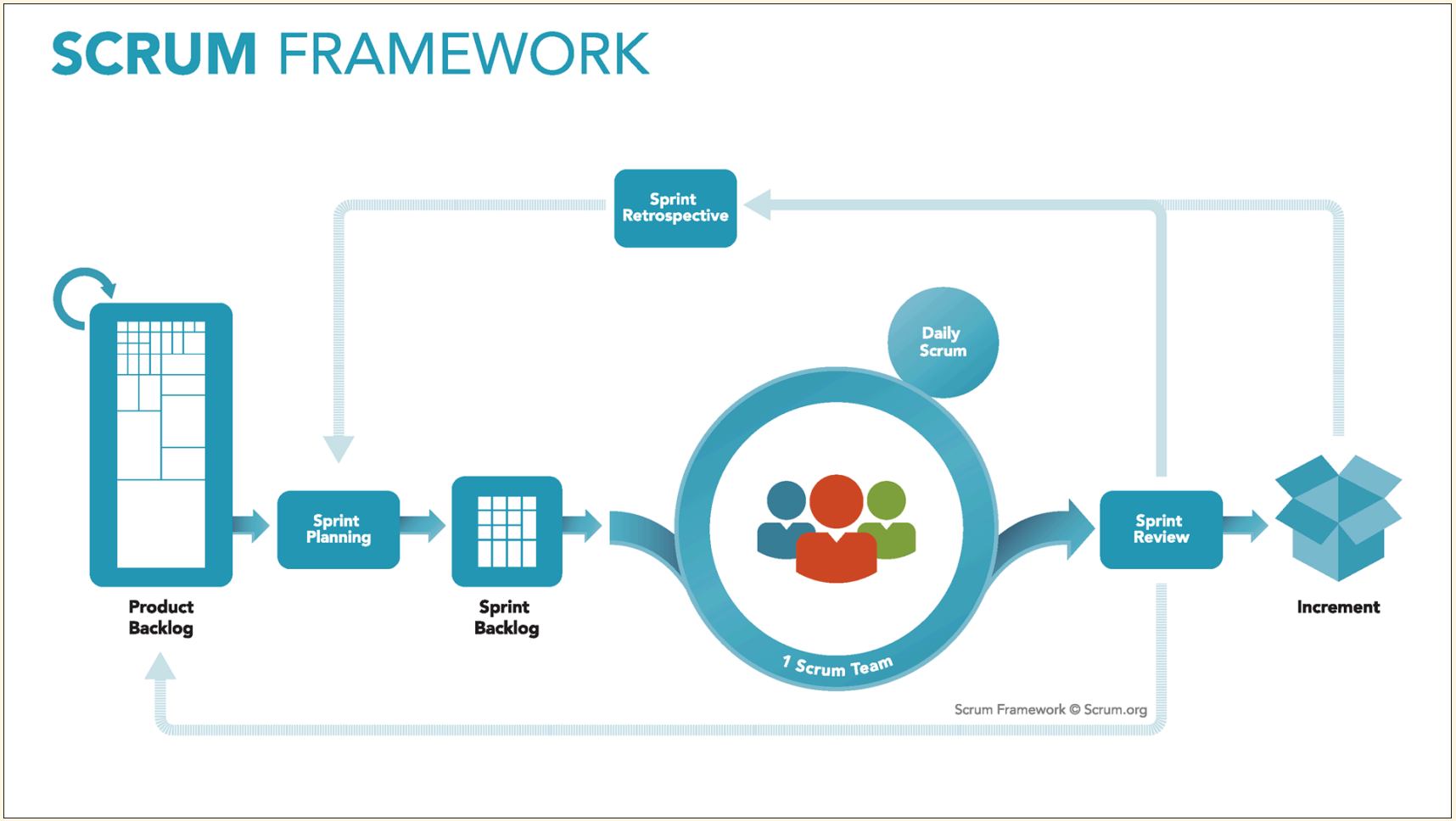
Un **issue** de GitHub es la forma básica de gestionar un proyecto en GitHub: una ficha con título, descripción, responsables y comentarios, asociada a un repositorio. Sirve para:

- Reportar errores (*bugs*)
- Solicitar funcionalidades
- Hacer preguntas
- Documentar tareas
- Discutir…

GITHUB PROJECTS

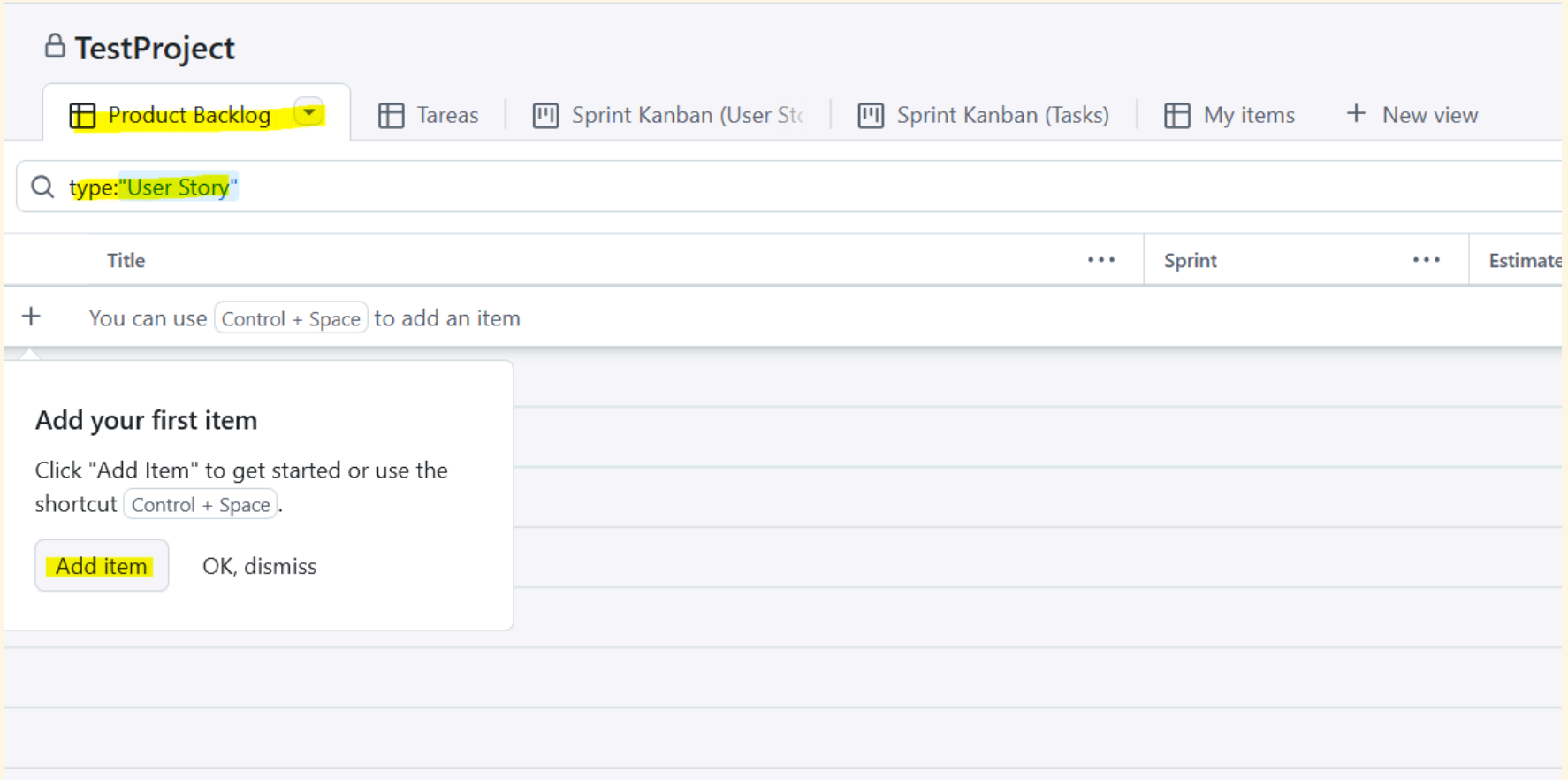
- Herramienta de **gestión de proyectos** integrada en GitHub
- Relativamente nueva (2024), así que todavía tiene mejoras pendientes
- Competencia clásica: Jira, Trello, Azure DevOps…
- Ventajas: se integra con los repositorios de GitHub y es accesible (gratuita, relativamente sencilla de usar)
- Funciona **sobre el sistema de issues**: cada ítem del proyecto es un issue
- Ofrece plantillas variadas y personalizables; como ninguna estaba orientada a Scrum, preparé una para el curso

MARCO SCRUM


SCRUM FRAMEWORK
Sprint Retrospective
Daily Scrum
Product Backlog
Sprint Planning
Sprint Backlog
1 Scrum Team
Sprint Review
Increment
Scrum Framework © Scrum.org

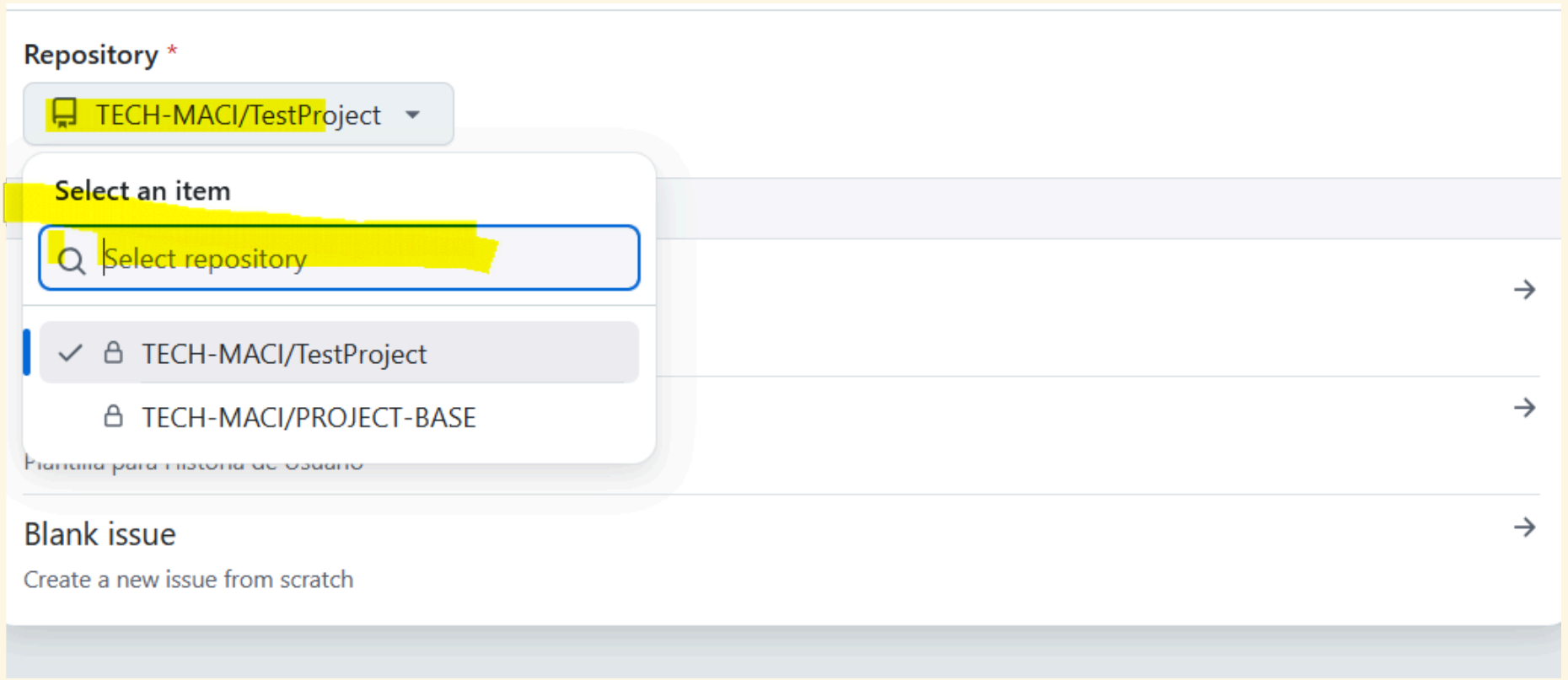
NUEVA HISTORIA DE USUARIO (1/4)

En la vista **Product Backlog** (filtrada por `type:"User Story"`), *Add item*:



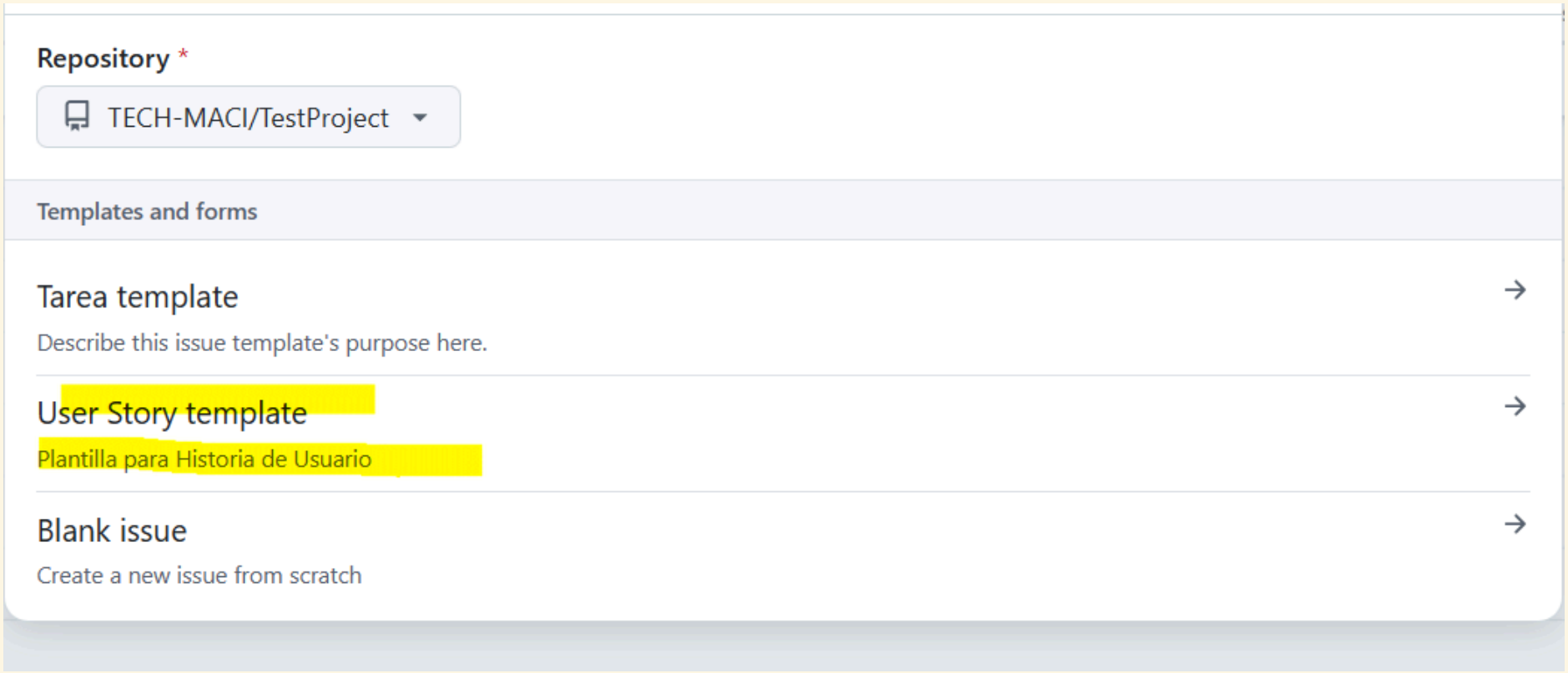
NUEVA HISTORIA DE USUARIO (2/4)

Elegir el repositorio del grupo:

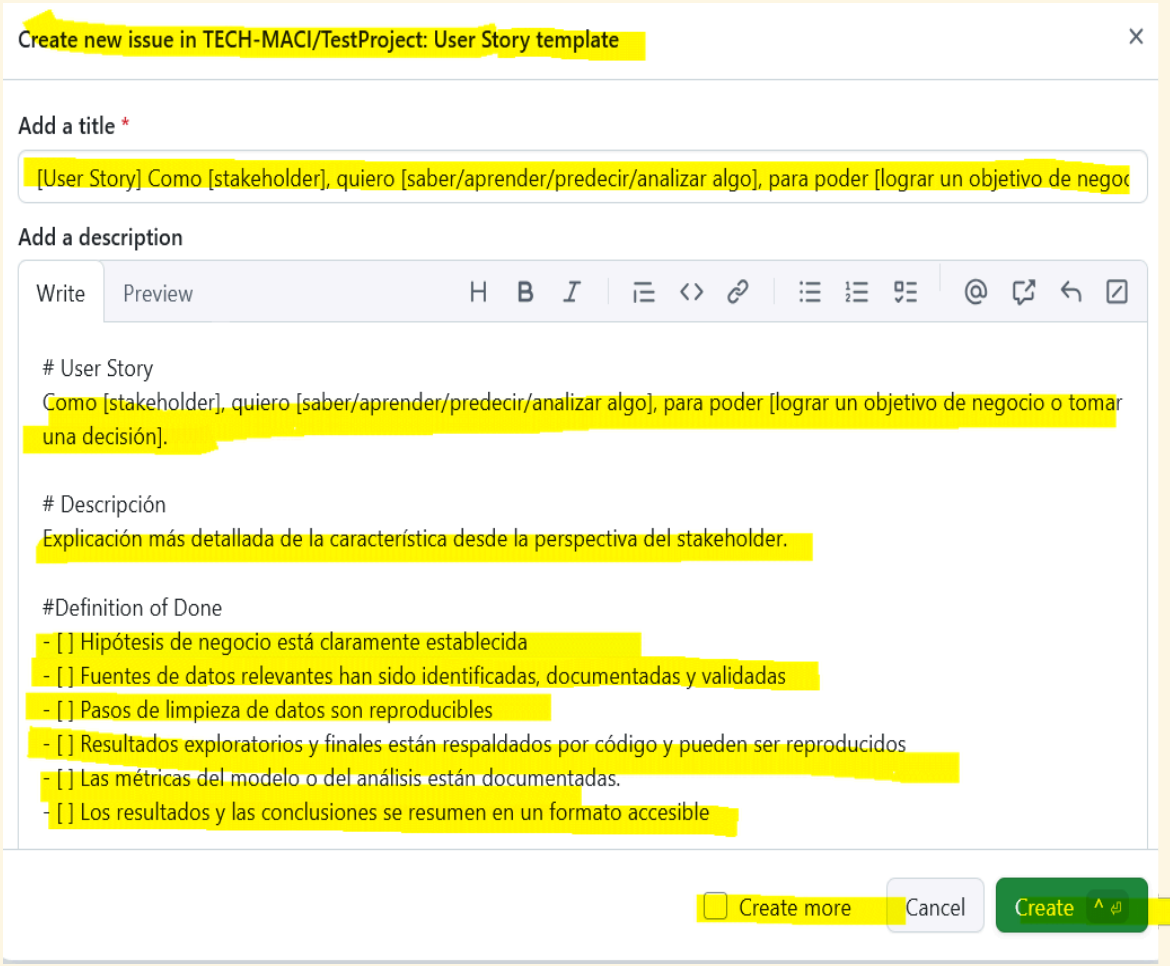


NUEVA HISTORIA DE USUARIO (3/4)

Elegir la plantilla **User Story template**:



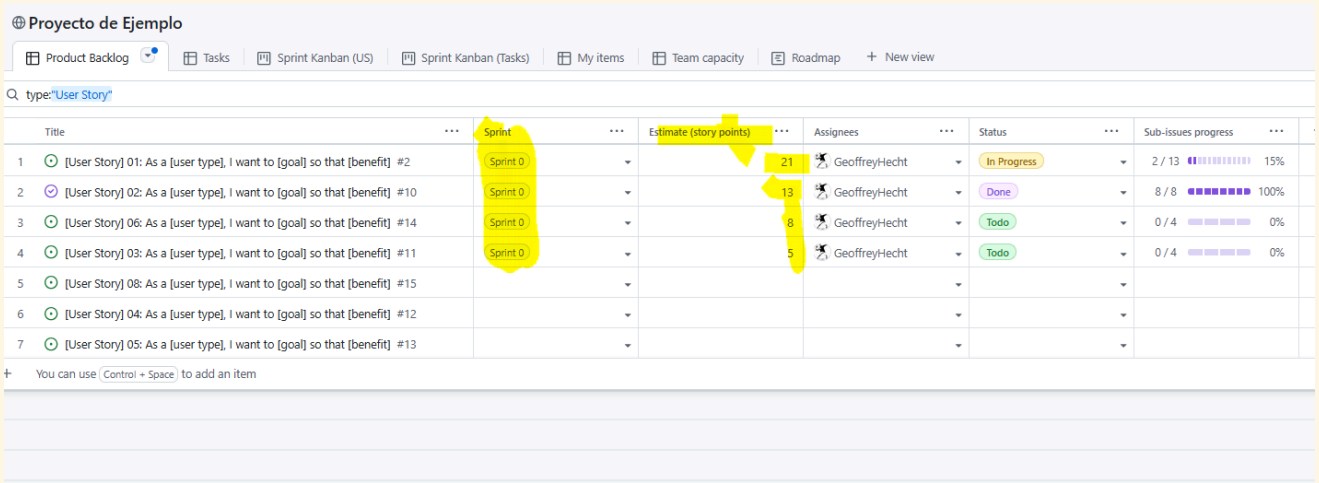
NUEVA HISTORIA DE USUARIO (4/4)



Completar la plantilla:

- **Título**: la historia en el formato “Como [stakeholder], quiero…, para poder…”
- **Descripción**: la característica desde la perspectiva del stakeholder
- **Definition of Done**: la lista de verificación común, ya incluida

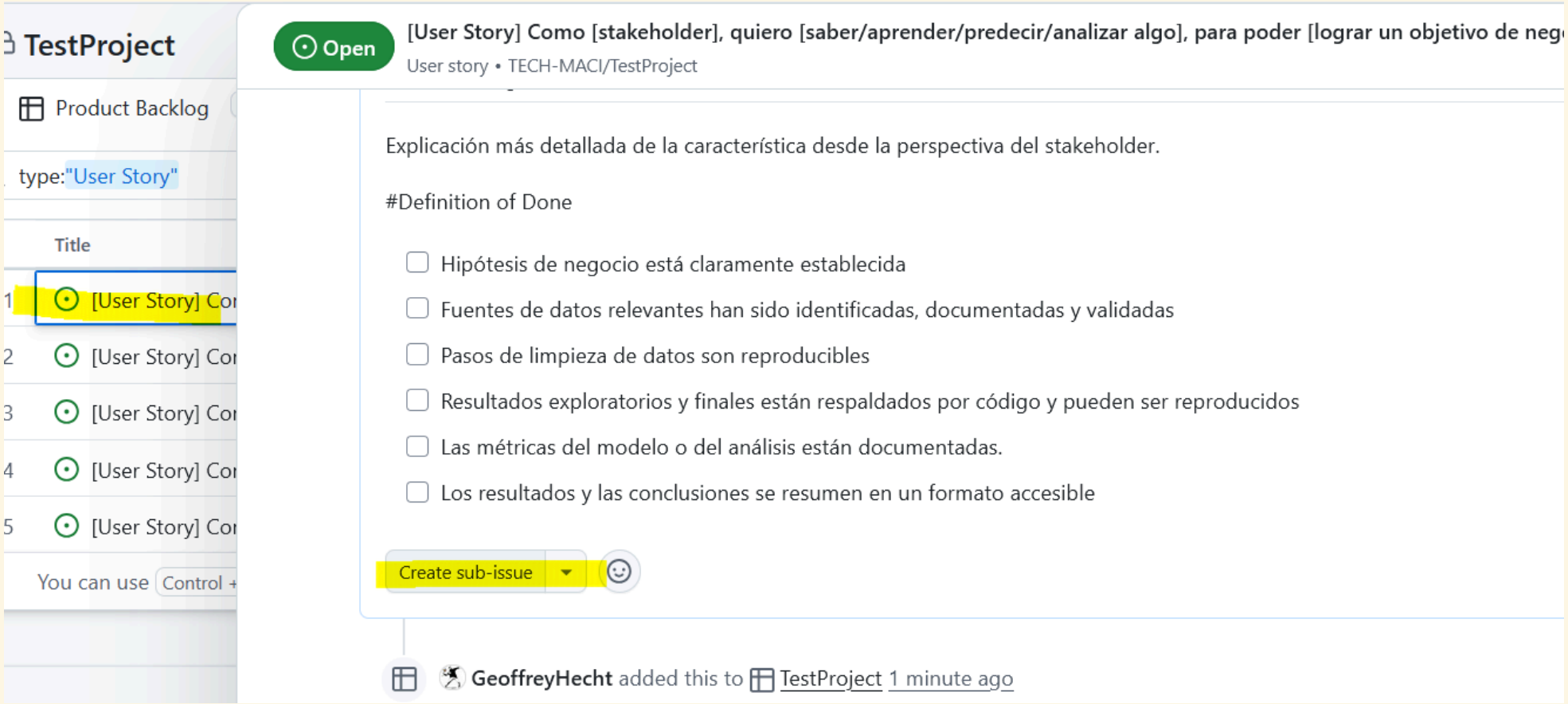
PRODUCT BACKLOG



- Todos participan en la elaboración de las historias de usuario, pero la responsabilidad recae en el **Product Owner**
- El PO ordena las historias por **prioridad**, las más urgentes al principio de la lista
- Luego el equipo hace la **estimación**, por ejemplo con Planning Poker
- Finalmente, el equipo decide qué historias incluir en el **sprint**

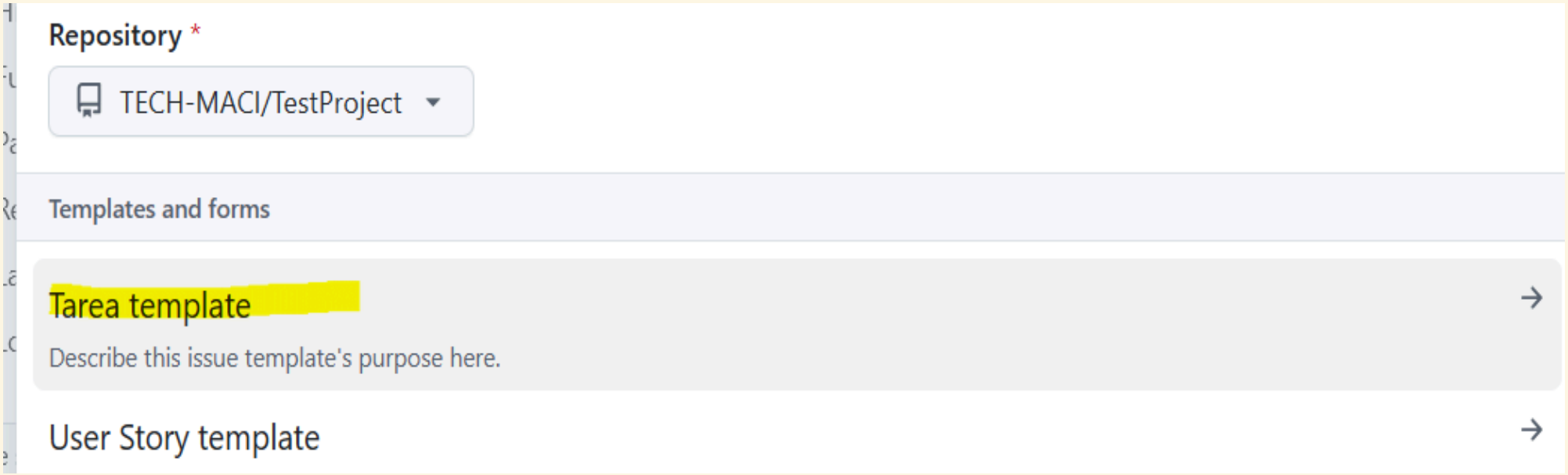
CREACIÓN DE TAREAS (1/3)

Desde una historia de usuario, *Create sub-issue*:

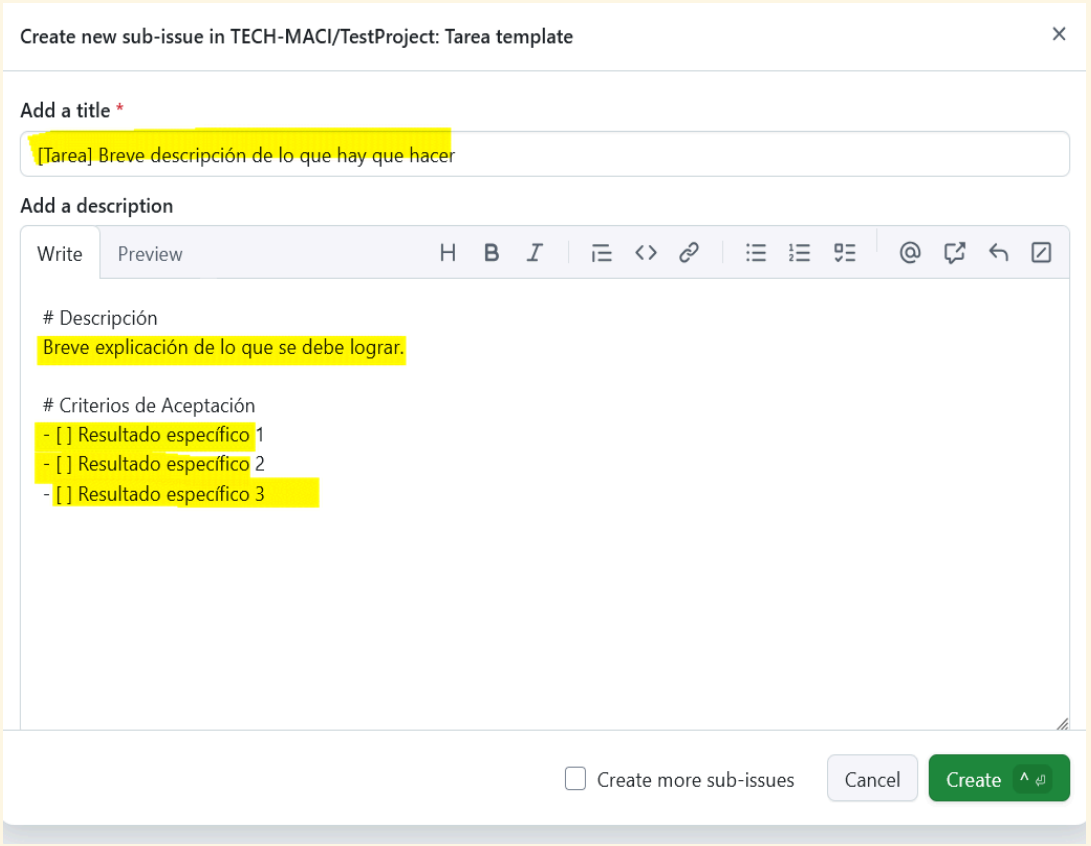


CREACIÓN DE TAREAS (2/3)

Elegir la plantilla **Tarea template**:



CREACIÓN DE TAREAS (3/3)

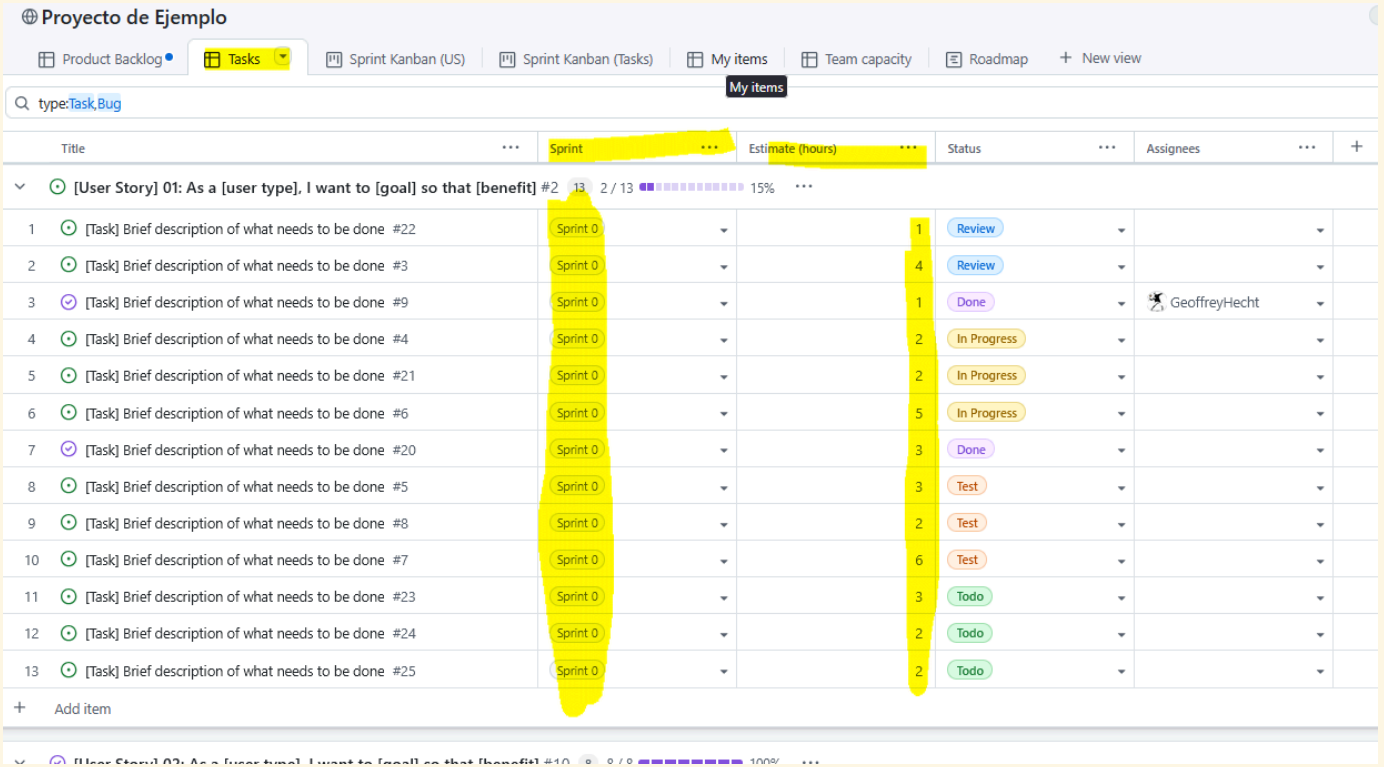


- **Título**: qué hay que hacer
- **Descripción**: qué se debe lograr
- **Criterios de aceptación**: resultados que indican que la tarea está terminada

...

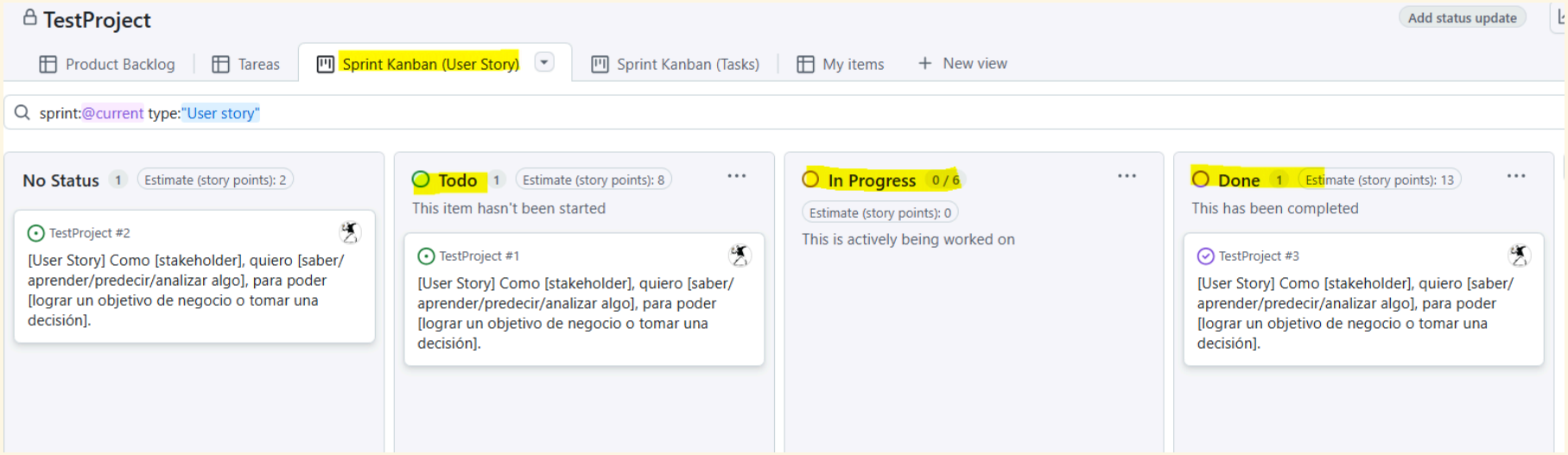
Las tareas las crea el **equipo de desarrollo**; cada una es necesaria para terminar una historia.

VISTA DE TAREAS



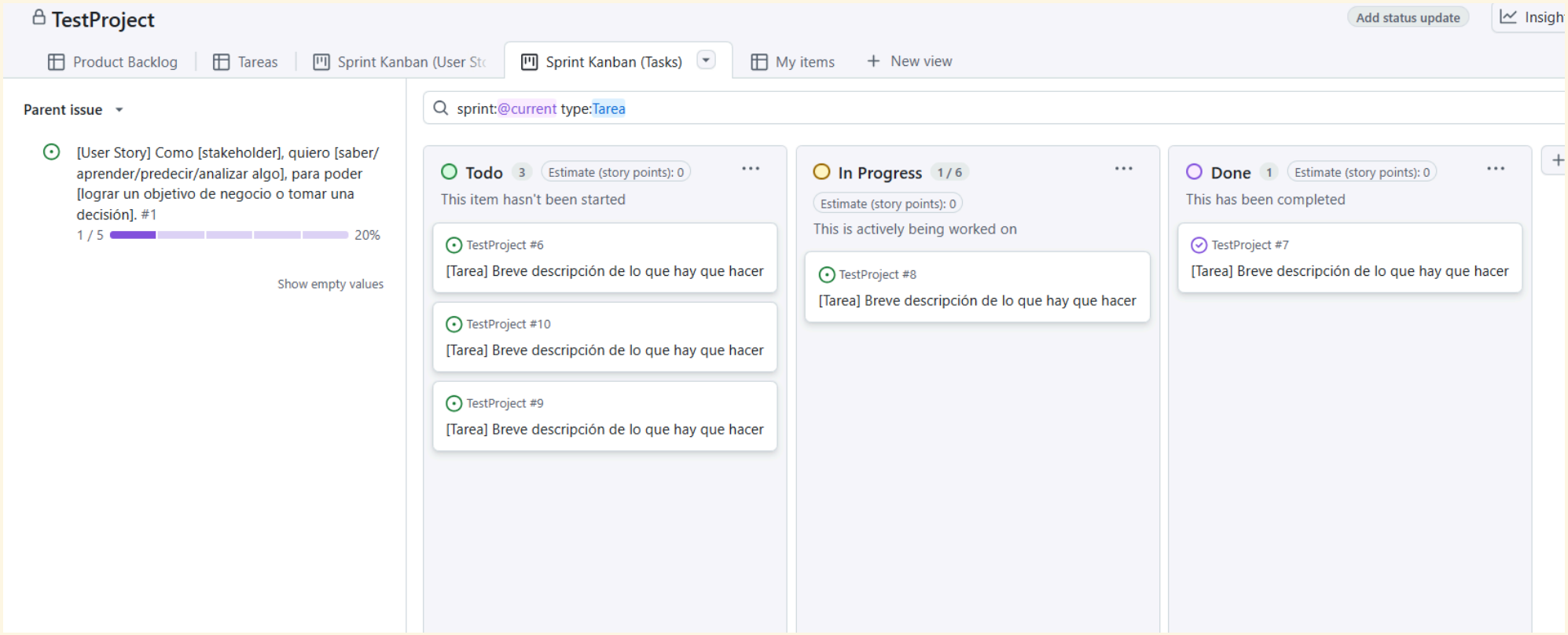
- Asignar el **sprint** a cada tarea creada: GitHub no lo hereda de la historia, aunque sea una subtarea
- Estimar el **número de horas** de cada tarea

SPRINT KANBAN DE HISTORIAS



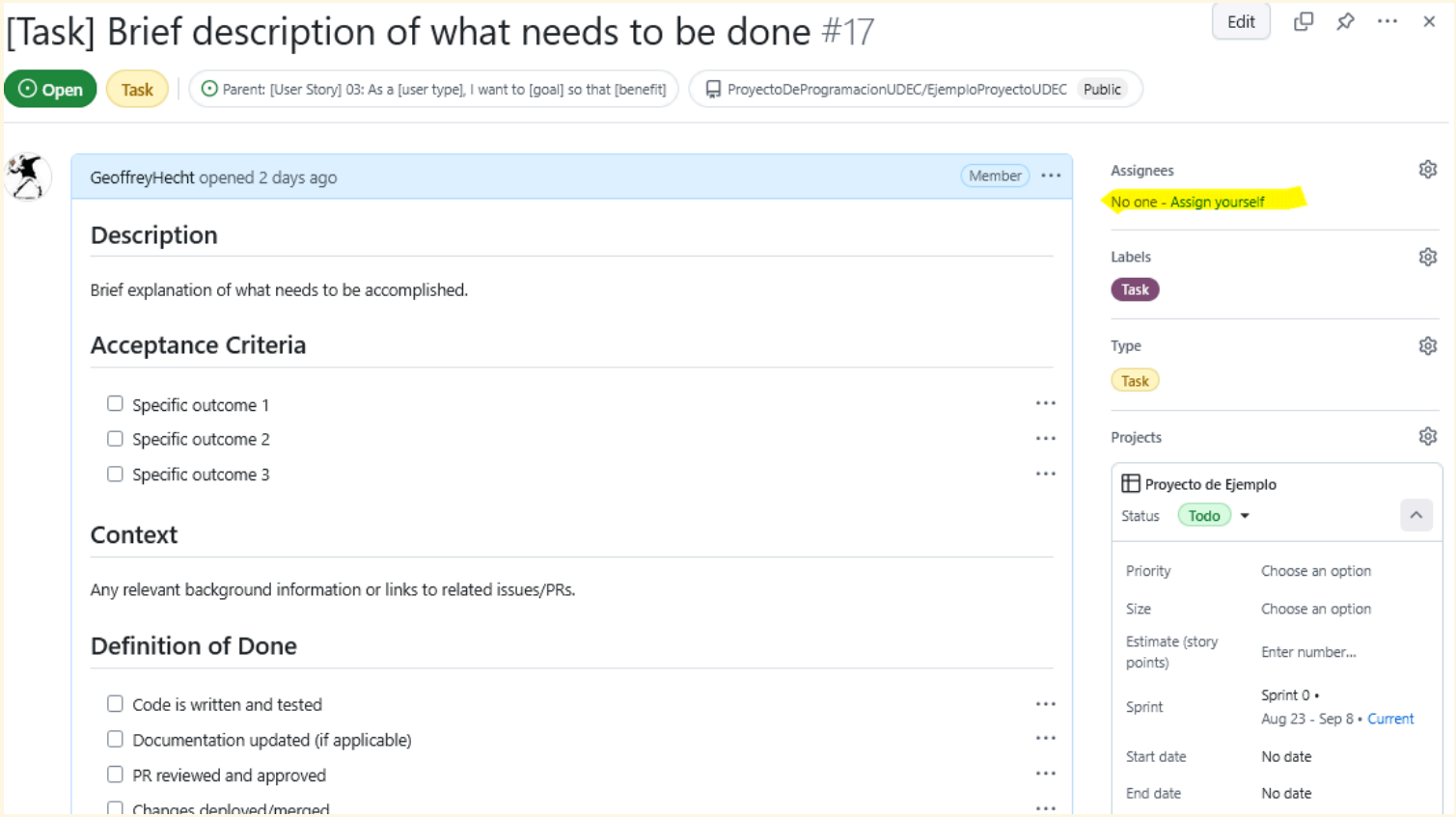
- Esta vista sirve para actualizar el estado de las historias de usuario: es la vista principal del PO, pero los desarrolladores la mantienen al día
- Al principio todo estará en *No Status*: hay que pasarlo a *Todo* y actualizar el resto a medida que avanza el sprint

SPRINT KANBAN DE TAREAS



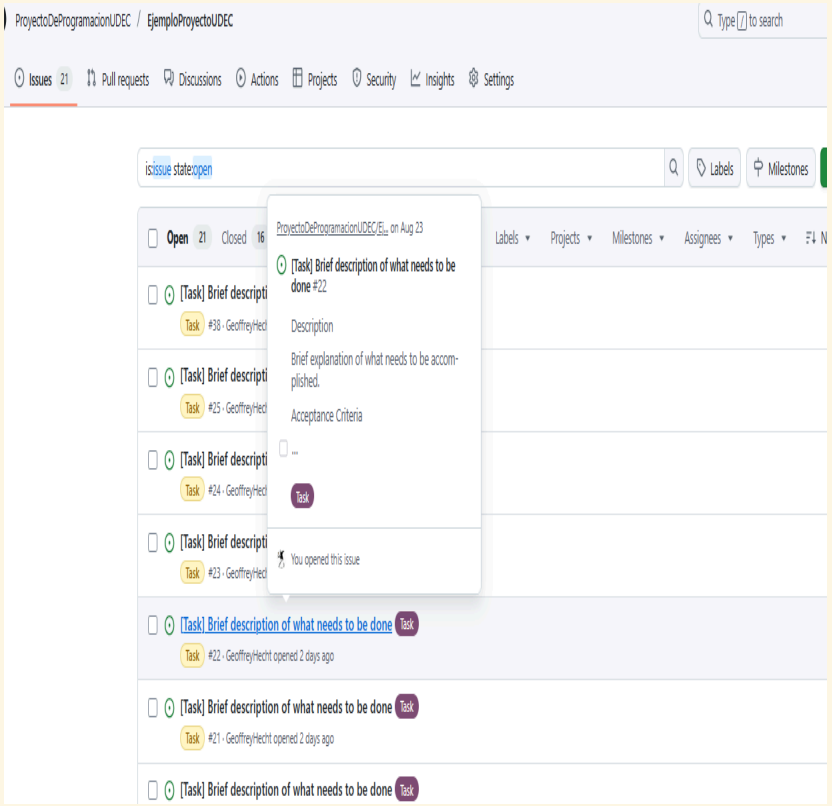
Lo mismo para las tareas: conviene terminarlas una por una, en lugar de avanzar varias en paralelo.

DETALLE DE UNA TAREA



Al abrir una tarea (o una historia de usuario) se puede actualizar su estado y asignarla, a usted o a otra persona. Esta información debe estar siempre al día.

ISSUES EN EL REPOSITORIO



- Los issues también están disponibles, y se pueden gestionar, desde el repositorio
- Un issue se puede cerrar (marcar como completado) desde un commit o una Pull Request, con una palabra clave (`close`, `closes`, `fix`, `fixes`, `resolve`...) seguida de la referencia del issue:

```
closes #18                          # mismo
repositorio
closes TECH-MACI/2026-G02#18        # otro
repositorio
```

Documentación: vincular una PR a un issue

ROLES EN SCRUM

Recordatorio de la clase de métodos ágiles:

Rol	Responsabilidades
Product Owner	Gestiona el Product Backlog, define la visión del producto, acepta o rechaza el trabajo según los criterios de aceptación, representa al cliente
Scrum Master	Facilita el proceso, acompaña la mejora continua y resuelve impedimentos, protege al equipo de distracciones externas
Equipo de desarrollo	Entrega incrementos que cumplen la DoD, se autoorganiza y estima, colabora para alcanzar el objetivo del sprint

En este curso, el PO y el Scrum Master también forman parte del equipo de desarrollo.

ROLES EN EL CURSO

Product Owner

- Responsable del backlog: traza las líneas generales y establece las prioridades
- Mantiene la comunicación con el cliente (el equipo docente)
- Sigue el avance de las historias de usuario
- Confirma el *Done* de las historias de usuario
- Puede ayudar un poco con el desarrollo

Scrum Master

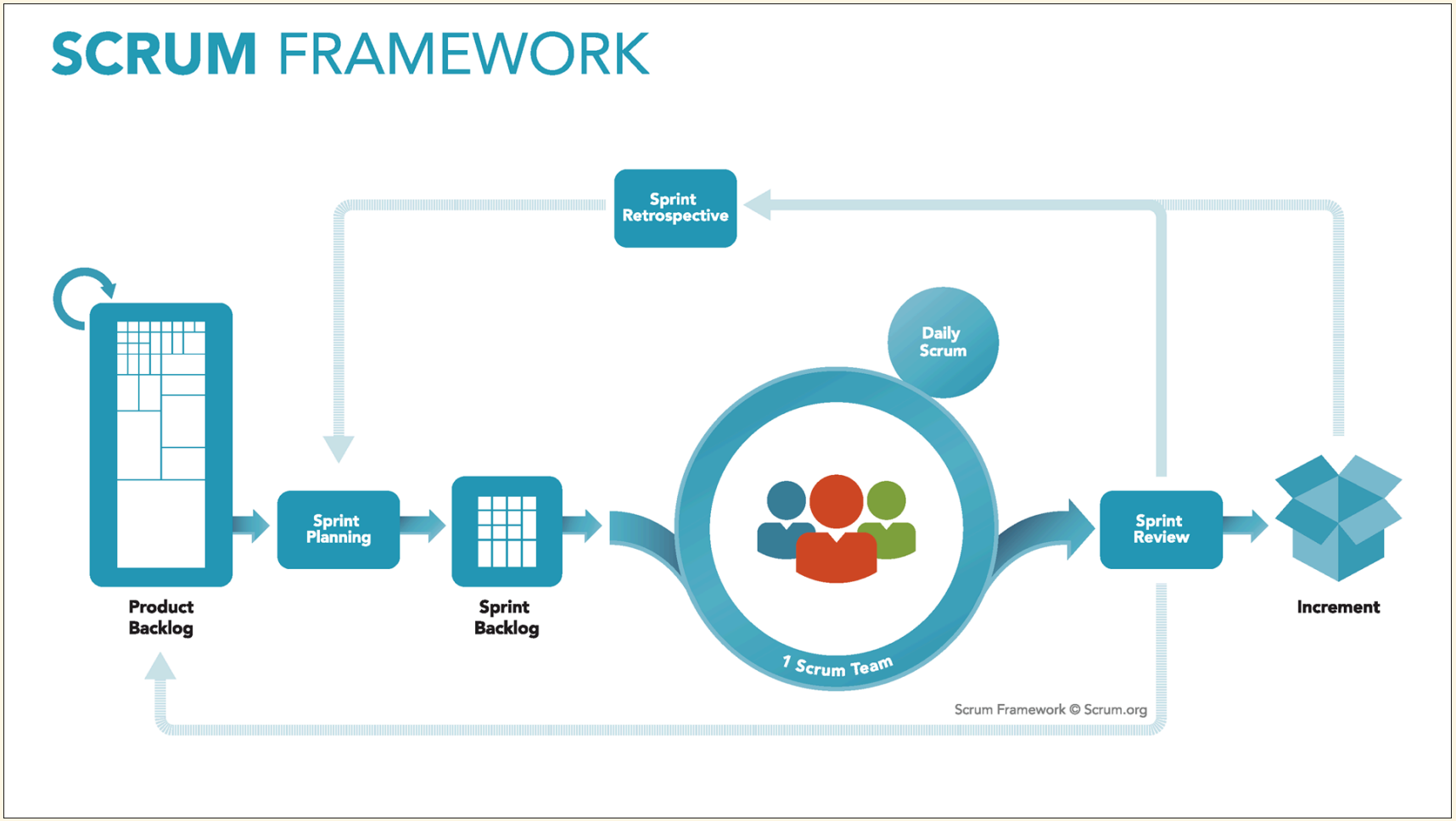
- Organiza las reuniones Scrum
- Vela por que se siga la metodología y se use bien GitHub Projects
- Apoya al resto del equipo
- Ayuda a estimar

EQUIPO DE DESARROLLO EN EL CURSO

- Participa en la formulación de las historias de usuario: estimación, Definition of Done, etc.
- Es responsable de crear las tareas y estimarlas
- Se autoorganiza para elegir las tareas, y mantiene el tablero Kanban actualizado
- Desarrolla las tareas con la mejor calidad posible

Los roles cambian en cada sprint.

MARCO SCRUM


SCRUM FRAMEWORK
Sprint Retrospective
Daily Scrum
Product Backlog
Sprint Planning
Sprint Backlog
1 Scrum Team
Sprint Review
Increment
Scrum Framework © Scrum.org

EVALUACIÓN: EVENTOS DEL SPRINT

Criterio	Excelente	Bien	Por mejorar	Falta (0)
Sprint review	El trabajo presentado corresponde a un incremento completo y permite una demostración a los stakeholders. La presentación es clara y estructurada	El trabajo corresponde a un incremento completo y permite una demostración, pero la presentación carece de estructura o claridad	El trabajo no está lo suficientemente terminado para demostrar un incremento completo, pero los avances permiten obtener un mínimo de retroalimentación de los stakeholders	El trabajo presentado es insuficiente para demostrar un avance concreto del proyecto
Sprint retrospective	La síntesis es exhaustiva y evidencia una reflexión profunda del equipo. Resume de manera concisa lo que funcionó, lo que requiere mejoras y las resoluciones para el próximo sprint	La síntesis es globalmente completa y demuestra una reflexión pertinente sobre el proceso, pero carece de detalles en ciertos aspectos	La síntesis es limitada; la reflexión sobre el proceso es superficial o incompleta	La retrospectiva no es relevante: no propone una reflexión sobre el proceso ni líneas de mejora

EVALUACIÓN: HERRAMIENTAS

Criterio	Excelente	Bien	Por mejorar	Falta (0)
Uso de Git	Commits frecuentes, descriptivos y bien estructurados. Los mensajes explican la razón del commit y no se limitan a describir los cambios	Commits regulares y descriptivos en la mayoría de los casos, pero algunos mensajes podrían ser más informativos y precisos	Uso moderado de Git, con problemas recurrentes en los commits	Uso de Git insuficiente o que no respeta las recomendaciones vistas en clase
Uso de GitHub Projects y Scrum	Se usan correctamente todas las funcionalidades vistas en clase (issues, tablero Kanban, plantillas…). La implementación de Scrum es evidente en la organización del trabajo	Se usa GitHub Projects adecuadamente, pero faltan algunos elementos. Scrum se aplica de manera básica pero funcional	El uso de GitHub Projects es limitado. La implementación de Scrum es inconsistente	No se usa GitHub Projects correctamente o no se evidencia Scrum

Excelente, Bien y Por mejorar corresponden al tercio superior, medio e inferior de la escala.

EVALUACIÓN: PLANIFICACIÓN Y EQUIPO

Criterio	Excelente	Bien	Por mejorar	Falta (0)
Gestión de issues y planificación	Issues bien definidos, categorizados y actualizados constantemente. Planificación clara y detallada, que refleja una comprensión sólida de la gestión de proyectos. Sprints bien estructurados, con objetivos claros	Issues definidos y actualizados regularmente. Planificación adecuada. Sprints con estructura, pero con claridad mejorable	Issues presentes pero con definición irregular. Planificación básica. Sprints sin estructura clara	Issues escasos o mal definidos. Sin evidencia clara de planificación. Sprints no estructurados
Trabajo en equipo y liderazgo	Excelente colaboración, visible en los commits y en GitHub Projects. Liderazgo efectivo y distribución equilibrada del trabajo. Comunicación clara y constructiva en los issues	Buena colaboración, con participación regular de todos. Liderazgo presente pero mejorable. Comunicación adecuada	Colaboración básica, con participación irregular. Liderazgo limitado. Comunicación mínima pero funcional	Poca evidencia de colaboración. Sin liderazgo. Comunicación deficiente o inexistente

PRÓXIMA SEMANA

- Desde ahora comienza el **sprint 1**
- Tener el backlog listo para el sprint 1 y haber comenzado las primeras tareas
- Hacer algunas reuniones diarias de Scrum durante la semana

REFERENCIAS

- Managing your projects with GitHub
- GitHub Docs: About Projects
- Benday: GitHub Projects